

GUÍA DE PLANIFICACIÓN

PROYECTO FINAL

Define tu idea antes de escribir una sola línea de código

HTML · CSS · JavaScript · APIs

Desarrollo Web Frontend

ÍNDICE

Introducción	¿Por qué planificar antes de programar?
Paso 1	Definición de la idea
Paso 2	Definición del MVP
Paso 3	Alcance del proyecto
Paso 4	Definición técnica básica
Paso 5	Diseño en papel (Wireframe)
Resumen	Checklist final

INTRODUCCIÓN

¿Por qué planificar antes de programar?

Imagina que quieres construir una casa. ¿Empezarías poniendo ladrillos sin tener un plano? Probablemente no. Lo mismo pasa con el desarrollo de software: si empiezas a programar sin saber exactamente qué vas a construir, terminarás perdido, rehaciendo todo o abandonándolo.

Esta guía te llevará paso a paso para que defines tu proyecto **en papel** antes de tocar el código. Al final, tendrás una base sólida que te servirá como mapa para desarrollar tu aplicación web.

Objetivo de esta guía

Al terminar esta guía tendrás:

- Una idea clara y bien definida
- Un MVP (versión mínima) realista
- Límites claros de lo que incluirás y lo que no
- Las tecnologías que usarás
- Un boceto visual de tu aplicación

Requisitos previos: Conocimientos básicos de HTML, CSS y JavaScript. Familiaridad inicial con el consumo de APIs.

01 DEFINICIÓN DE LA IDEA

Antes de pensar en código, necesitas responder una pregunta fundamental: **¿Qué vas a construir y por qué?**

Una buena idea de proyecto no tiene que ser revolucionaria. Tiene que ser **clara**, **resolverle un problema real a alguien** y ser **alcanzable** con tus conocimientos actuales.

Las 4 preguntas clave

Tu idea debe poder explicarse respondiendo estas cuatro preguntas. Si no puedes responderlas de forma sencilla, tu idea aún no está suficientemente definida.

Pregunta	Significado	Ejemplo
¿Qué es?	Describe tu proyecto en una frase	Una app web para gestionar tareas diarias
¿Para qué sirve?	Qué función cumple, qué hace concretamente	Permite crear, organizar y completar tareas
¿Para quién es?	Quién lo usaría: tipo de usuario	Estudiantes que necesitan organizarse
¿Qué problema resuelve?	La necesidad real que atiende	Olvidar entregas y no tener un sistema simple

■ Ejemplo: App del Clima

¿Qué es? Una aplicación web que muestra el clima actual de cualquier ciudad. **¿Para qué sirve?** Para consultar temperatura, humedad y pronóstico rápidamente. **¿Para quién?** Cualquier persona que necesite saber el clima antes de salir. **¿Qué problema resuelve?** No tener que buscar en múltiples sitios; información rápida y visual.

■ Ejemplo: Buscador de Películas

¿Qué es? Una web que permite buscar películas y ver sus detalles. **¿Para qué sirve?** Para encontrar información sobre películas (sinopsis, calificación, póster). **¿Para quién?** Amantes del cine que quieren decidir qué ver. **¿Qué problema resuelve?** Tener un catálogo visual rápido sin publicidad invasiva.



Preguntas para responder

- ¿Puedes explicar tu proyecto en UNA sola frase?
- ¿Qué hace concretamente tu aplicación? (no qué "podría hacer", sino qué HACE)
- ¿Quién sería el usuario típico? Descríbelo.
- ¿Qué problema específico le resuelve a esa persona?
- ¿Existe algo similar? ¿Qué harías diferente o más simple?

■ Error común a evitar

Idea demasiado vaga: "Quiero hacer algo con APIs" o "una red social". Si no puedes responder las 4 preguntas clave en 30 segundos, tu idea necesita más trabajo. Sé específico: no digas "algo para organizar", di "una lista de tareas con categorías y fechas límite".

■ Error común a evitar

Elegir una idea solo porque "se ve cool": Si no te interesa el problema que resuelve, perderás la motivación a mitad del proyecto. Elige algo que te importe o que te gustaría usar tú mismo.

02 DEFINICIÓN DEL MVP

MVP significa **Producto Mínimo Viable** (Minimum Viable Product). Es la versión más simple de tu proyecto que funciona y es útil. No es el proyecto completo, es el corazón del proyecto.

Piensa en el MVP como la respuesta a esta pregunta: **¿Cuál es la versión más básica de mi proyecto que todavía resolvería el problema principal?**

¿Por qué es importante definir un MVP?

- **Evita la parálisis:** Si piensas en todas las funciones posibles, nunca empezarás.
- **Te da un objetivo claro:** Sabes exactamente qué construir primero.
- **Permite iterar:** Puedes agregar mejoras después de que lo básico funcione.
- **Es realista:** Se ajusta al tiempo y conocimientos que tienes ahora.

Cómo definir tu MVP en 3 pasos

1. **Lista TODAS las funcionalidades** que imaginas para tu proyecto (sueña en grande).
2. **Clasifica cada una** en: Esencial (sin esto no funciona), Importante (mejora la experiencia) o Bonus (sería genial pero no es necesario).
3. **Tu MVP = solo las Esenciales.** Todo lo demás lo agregas después.

Ejemplo: Buscador de Películas

Funcionalidad	Categoría	¿En el MVP?
Buscar películas por nombre	Esencial	Sí
Ver póster y sinopsis	Esencial	Sí
Diseño responsive básico	Esencial	Sí
Guardar favoritos en localStorage	Importante	No (fase 2)
Filtrar por género o año	Importante	No (fase 2)
Modo oscuro	Bonus	No (fase 3)
Crear cuenta de usuario	Bonus	No (fase 3)

Compartir en redes sociales	Bonus	No (fase 3)
-----------------------------	-------	-------------

Preguntas para responder

- ¿Cuáles son las 3-5 funcionalidades SIN LAS CUALES tu proyecto no tiene sentido?
- ¿Qué funcionalidades son "nice to have" pero no imprescindibles?
- Si solo tuvieras 3 días para entregar algo funcional, ¿qué incluirías?
- ¿Tu MVP resuelve el problema principal que definiste en el Paso 1?

■ Error común a evitar

Querer hacerlo todo desde el inicio: El error número 1 de los estudiantes es incluir demasiadas funcionalidades en la primera versión. Login, dashboard, modo oscuro, animaciones... Empieza con lo mínimo. Un proyecto que funciona bien con 3 features es mejor que uno roto con 10.

■ Consejo

Si tu MVP tiene más de 5 funcionalidades esenciales, probablemente estás incluyendo cosas que no son realmente esenciales. Pregunta: ¿Sin ESTO, mi app literalmente no sirve? Si la respuesta es no, sacálo del MVP.

03 ALCANCE DEL PROYECTO

Definir el alcance significa trazar una línea clara entre **lo que SÍ harás** y **lo que NO harás**. Es como poner una cerca alrededor de tu proyecto para que no crezca sin control.

El mayor enemigo de un proyecto estudiantil es el **scope creep**: ir agregando cosas mientras desarrollas. "Ah, y también podría agregar esto..." es la frase que mata proyectos.

Cómo definir el alcance

Crea dos listas muy claras. La primera es todo lo que tu proyecto **incluirá** (funcionalidades, páginas, interacciones). La segunda es todo lo que **explícitamente NO incluirá**, aunque alguien podría esperarlo.

Ejemplo: Buscador de Películas

INCLUYE	NO INCLUYE
✓ Página de inicio con buscador ✓ Resultados en tarjetas con imagen ✓ Página de detalle por película ✓ Diseño responsive (mobile first) ✓ Consumo de API OMDb ✓ Manejo de errores básico	✗ Login / registro de usuarios ✗ Base de datos o backend propio ✗ Reproducción de trailers ✗ Sistema de reseñas o comentarios ✗ Notificaciones push ✗ Integración con redes sociales

Cantidad de páginas o vistas

Define cuántas páginas o secciones tendrá tu proyecto. Para un MVP, entre **1 y 3 páginas** suele ser suficiente. Cada página adicional suma trabajo de diseño, maquetación y lógica.

Tipo de proyecto	Páginas sugeridas (MVP)
App de tareas (To-Do)	1 página (todo en una vista)
Buscador con API	2 páginas (búsqueda + detalle)
Dashboard informativo	1-2 páginas
Portafolio personal	1 página (single page) o 3 secciones

Clon simple (Spotify, etc.)	2-3 páginas (inicio + reproductor + búsqueda)
-----------------------------	---

Preguntas para responder

- *Lista exactamente qué funcionalidades INCLUYE tu proyecto.*
- *Lista al menos 3 cosas que tu proyecto NO incluirá (aunque alguien podría esperarlas).*
- *¿Cuántas páginas o vistas tendrá tu MVP?*
- *¿Cada funcionalidad incluida es realmente necesaria para resolver el problema principal?*

■ Error común a evitar

No definir qué NO incluye: Si no pones límites explícitos, tu proyecto crecerá sin control. Escribe la lista de exclusiones y respétala. Si más adelante quieres agregar algo, hazlo solo después de que tu MVP esté 100%% funcionando.

04 DEFINICIÓN TÉCNICA BÁSICA

Ahora que sabes QUÉ vas a construir, necesitas definir CÓMO lo vas a construir. No necesitas ser experto en todas las tecnologías, pero sí debes saber cuáles usarás y para qué.

Tecnologías base

Capa	Tecnología	Para qué
Estructura	HTML5	Construir la estructura de cada página
Estilos	CSS3 (o framework como Bootstrap/Tailwind)	Diseño visual, responsive design
Lógica	JavaScript (vanilla)	Interactividad, lógica de negocio, DOM
Datos externos	API REST (fetch)	Obtener datos dinámicos de servicios externos
Almacenamiento local	localStorage	Guardar preferencias o datos del usuario

APIs gratuitas recomendadas para proyectos

Usar una API real le da mucho valor a tu proyecto de portafolio. Aquí tienes opciones gratuitas y fáciles de implementar:

API	Qué ofrece	URL
OMDb API	Películas y series	omdbapi.com
OpenWeather	Clima actual y pronóstico	openweathermap.org
PokeAPI	Datos de Pokémon	pokeapi.co
REST Countries	Info de países del mundo	restcountries.com
The Cocktail DB	Recetas de cocteles	thecocktaildb.com
JSONPlaceholder	Datos falsos para practicar	jsonplaceholder.typicode.com

Dog CEO API	Imágenes de perros por raza	dog.ceo/dog-api
News API	Noticias de todo el mundo	newsapi.org

Estructura de archivos sugerida

Organiza tu proyecto desde el inicio. Una buena estructura hace que el código sea más fácil de entender y mantener.

Estructura de carpetas

```
mi-proyecto/
  ■■■ index.html
  ■■■ detalle.html    (si tienes segunda página)
  ■■■ css/
  ■   ■■■ styles.css
  ■■■ js/
  ■   ■■■ app.js      (lógica principal)
  ■   ■■■ api.js      (llamadas a la API)
  ■■■ img/            (imágenes locales)
  ■■■ README.md       (descripción del proyecto)
```

Preguntas para responder

- ¿Qué tecnologías usarás? (HTML, CSS puro o framework, JS vanilla o librería)
- ¿Tu proyecto consume alguna API? ¿Cuál? ¿Ya verificaste que es gratuita y funciona?
- ¿Necesitas guardar datos localmente? (localStorage)
- ¿Cómo organizarás tus archivos? Dibuja tu estructura de carpetas.
- ¿La API que elegiste tiene límite de peticiones? ¿Necesitas API key?

■ Error común a evitar

No probar la API antes de empezar: Muchos estudiantes eligen una API sin verificar que realmente funciona, que los datos vienen como esperan o que es gratuita. Antes de elegir, haz una prueba en el navegador o con fetch en la consola.

■ Consejo

Separa tu código en archivos con responsabilidades claras. No pongas TODO en un solo archivo app.js de 500 líneas. Un archivo para la lógica de la API, otro para la manipulación del DOM, otro para utilidades.

05 DISEÑO EN PAPEL (WIREFRAME)

Un wireframe es un **boceto simple** de cómo se verá tu aplicación. No necesita ser bonito, necesita ser útil. Es como el plano de un arquitecto: muestra dónde va cada cosa.

Puedes hacerlo en papel con lápiz, en una pizarra, o en herramientas gratuitas como **Excalidraw** (excalidraw.com), **Figma** (figma.com) o **draw.io** (app.diagrams.net). Lo importante es que lo hagas ANTES de programar.

Qué incluir en tu wireframe

- **Encabezado (header):** Logo/nombre del proyecto, navegación si la hay.
- **Contenido principal:** Dónde van los datos, las tarjetas, los formularios de búsqueda.
- **Interacciones clave:** Dónde hace clic el usuario, qué pasa cuando busca algo.
- **Pie de página (footer):** Créditos, links, información adicional.
- **Estados especiales:** ¿Qué se ve cuando no hay resultados? ¿Qué se ve mientras carga?

Ejemplo de wireframe anotado

A continuación, un ejemplo de cómo podría verse un wireframe sencillo para un buscador de películas. Nota cómo cada sección tiene una etiqueta que explica su propósito:



Consejos para tu wireframe

- No te preocupes por colores ni tipografía exacta. Eso viene después.
- Usa rectángulos simples para representar imágenes y bloques de texto.
- Anota qué pasa cuando el usuario interactúa ("al hacer clic aquí, muestra X").
- Haz un wireframe por cada página o vista de tu proyecto.
- Si tu proyecto es responsive, haz un boceto para móvil y otro para desktop.

Preguntas para responder

- *¿Hiciste un boceto de cada página/vista de tu proyecto?*
- *¿Se entiende dónde irá cada elemento (header, buscador, resultados, footer)?*
- *¿Anotaste qué pasa con las interacciones principales?*
- *¿Consideraste los estados vacíos (sin resultados, error de API, cargando)?*
- *¿Alguien más puede entender tu boceto sin que tú se lo expliques?*

■ Error común a evitar

Saltar el wireframe y empezar a programar directo: "Ya lo tengo en la cabeza" es la frase famosa antes de rehacer el diseño 3 veces. 10 minutos dibujando te ahorran horas de código.

CHECKLIST FINAL

Antes de escribir tu primera línea de código, asegúrate de poder marcar todos estos puntos. Si alguno queda sin resolver, regresa al paso correspondiente.

IDEA

- Puedo explicar mi proyecto en una frase clara
- Sé para quién es y qué problema resuelve
- La idea me motiva y me interesa el problema

MVP

- Tengo máximo 3-5 funcionalidades esenciales definidas
- Separé lo esencial de lo "nice to have"
- Mi MVP resuelve el problema principal

ALCANCE

- Tengo una lista de lo que SÍ incluye mi proyecto
- Tengo una lista de lo que NO incluye
- Definí cuántas páginas/vistas tendrá el MVP

TÉCNICO

- Sé qué tecnologías voy a usar
- Si uso API, ya la probé y funciona
- Tengo clara la estructura de archivos

WIREFRAME

- Tengo un boceto de cada página/vista
- Se entienden las interacciones principales
- Consideré estados de carga, error y vacío

■ Consejo

Comparte tu planificación con un compañero antes de empezar a programar. Si esa persona entiende tu idea, tu MVP y tu wireframe sin que le expliques demasiado, significa que estás listo para comenzar.

Nota final

Recuerda: un proyecto bien planificado no solo es más fácil de desarrollar, también se ve mucho mejor en tu portafolio. Los reclutadores valoran que puedas explicar **por qué** tomaste cada decisión, no solo que el código funcione.

¡Ahora sí, a programar con un plan claro!